Tech Challenge — Fase 4

Microsserviços e Saga Pattern na Plataforma de Gestão de Oficina Mecânica

> **RASCUNHO PARA REVISÃO DO RAFAEL — NÃO ENVIAR AO PORTAL.** Revisão alinhada ao plano v9.2 vigente: o alvo de integração, CI e demonstração é Kind local/efêmero, conforme o rebaseline de 13/10/2026. Itens marcados **[PENDENTE]** ou **[CONFIRMAR]** ainda precisam de evidência verificável; este texto não declara execução, cobertura, qualidade, monitoramento ou aceite do avaliador. Prazo de entrega: 17/11/2026.

Integrantes: [CONFIRMAR] nome completo e RM de cada integrante (Fase 3: Rafael Xavier · Silas Furini).

Vídeo de demonstração (até 15 min): [PENDENTE] `<link YouTube/Vimeo>`. A demonstração será preparada no alvo local/efêmero Kind; o aceite do Kind como evidência de deploy Kubernetes pelo avaliador continua sendo um risco em aberto.

Repositórios (um por microsserviço):

Serviço	Repositório
OS Service (orquestrador da saga)	https://github.com/rafaelxvr/Tech-challenge-15SOAT
Billing Service (orçamento e pagamento)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-billing
Execution Service (diagnóstico e reparo)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-execution

Revisor: [CONFIRMAR] acesso do usuário `soat-architecture` nos três repositórios (na Fase 3 constava nos quatro daquela fase; não verificado para a Fase 4).

Sumário

1. Resumo executivo
2. Arquitetura dos microsserviços
3. Saga orquestrada
4. Divisão de responsabilidades e tecnologias
5. CI/CD, deploy local e evidências de Kind
6. Observabilidade (New Relic)
7. Qualidade e testes (SonarCloud)
8. Pagamento com Mercado Pago (sandbox)
9. Matriz de rastreabilidade do enunciado
10. Limitações e riscos conhecidos

1. Resumo executivo

O desenho da Fase 4 divide a aplicação da oficina em **três microsserviços independentes**, cada um com repositório, pipeline, infraestrutura de execução e banco próprios. Uma **saga orquestrada** coordena as transações que atravessam os três (abertura da OS, orçamento, aprovação, pagamento, reserva de peças e reparo) e registra compensações para falhas recuperáveis.

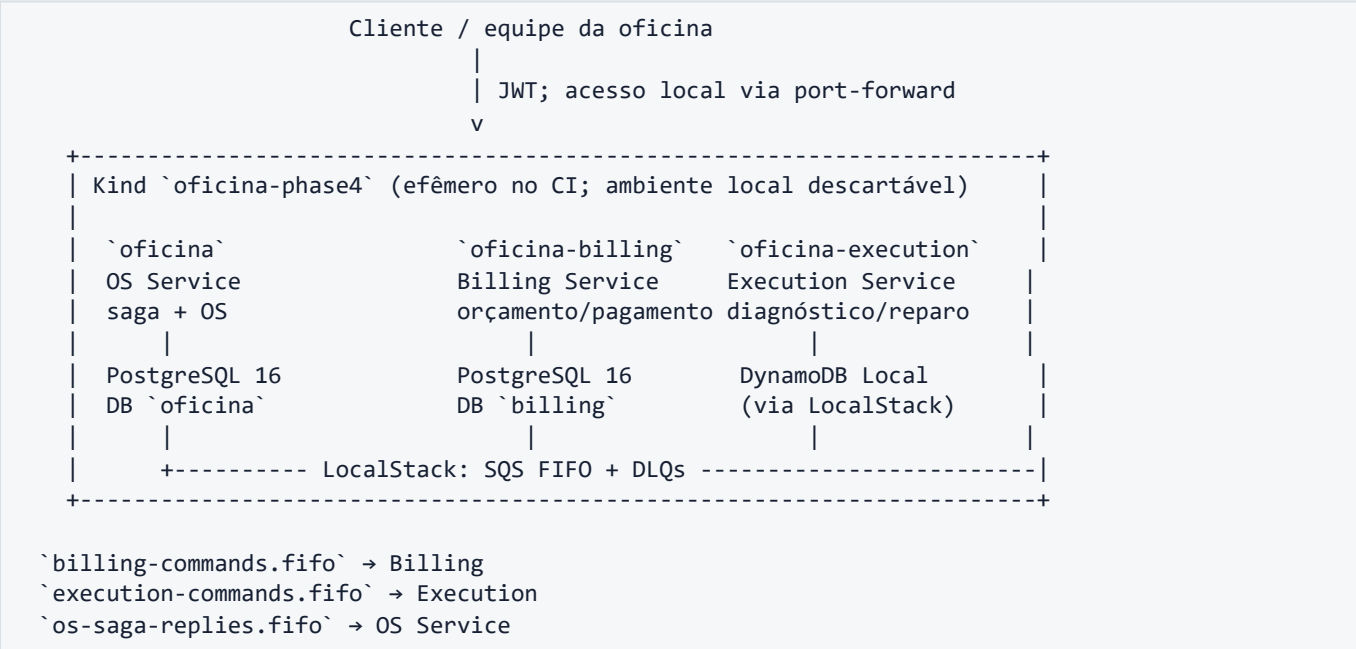
- **OS Service** é dono da ordem de serviço e orquestra a saga. Banco relacional PostgreSQL.
- **Billing Service** é dono do orçamento e do pagamento (Mercado Pago). Banco relacional PostgreSQL próprio.
- **Execution Service** é dono da fila de execução, do diagnóstico e do reparo. Banco não relacional DynamoDB.

Os serviços não acessam os bancos uns dos outros. A integração assíncrona usa filas SQS FIFO exercitadas localmente por LocalStack, com contratos JSON versionados. LocalStack é dependência de teste local; não representa provisionamento ou uso de uma conta AWS.

Situação deste rascunho: o alvo aprovado para integração e evidência de deploy é um cluster Kind local/efêmero. O estado dos códigos, pipelines, testes, deploys, observabilidade e integração com Mercado Pago deve ser confirmado pelos SHAs e resultados reais antes da submissão; este documento não os declara concluídos.

2. Arquitetura dos microsserviços

2.1 Diagrama geral



Diagramas Mermaid versionados: `docs/architecture/c4-*.md` no repositório do OS Service e o diagrama do README do Billing Service. **[PENDENTE]** revisar o diagrama geral contra os nomes e manifestos usados no alvo Kind, sem apresentá-lo como evidência de deploy.

2.2 Bancos isolados

Serviço	Banco no alvo local	Tipo	Justificativa
OS Service	PostgreSQL 16, banco `oficina`	SQL	OS, histórico e tabelas da saga precisam de atomicidade entre linhas

Serviço	Banco no alvo local	Tipo	Justificativa
Billing Service	PostgreSQL 16 próprio, banco `billing`	SQL	Orçamento, pagamento, inbox e outbox precisam compartilhar transações; confere valor e moeda
Execution Service	DynamoDB Local via LocalStack	NoSQL	Fila e estado por OS usam acesso por chave e escritas condicionais atômicas

Nenhum serviço lê ou escreve no banco de outro. Dados de outro domínio só chegam por mensagem.

2.3 Mensageria e contratos

Fila local	Dona (consumidor)	Conteúdo
`billing-commands.fifo` e `billing-commands-dlq.fifo`	Billing	`GerarOrcamento`, `RegistrarDecisao`, `CancelarOrcamento`, `EstornarPagamento`
`execution-commands.fifo` e `execution-commands-dlq.fifo`	Execution	`IniciarDiagnostico`, `IniciarReparo`, `CancelarExecucao`
`os-saga-replies.fifo` e `os-saga-replies-dlq.fifo`	OS Service	respostas de Billing e Execution

- Envelope comum: `messageId`, `kind`, `type`, `schemaVersion`, `sagaId`, `osId`, `passo`, `causationId`, `occurredAt`, `correlationId`, `traceparent` e `payload`; tamanho máximo de 65.536 bytes.
- `MessageGroupId = osId` e `MessageDeduplicationId = messageId`. Cada resposta copia os identificadores de correlação do comando; `causationId` aponta para o `messageId` respondido.
- Os contratos canônicos são JSON Schemas `contracts/<tipo>/v1.json`: o consumidor é dono do schema de comando e o produtor é dono do schema de resposta. O OS Service fixa cópias de respostas para testes de contrato.
- Inbox idempotente e outbox são persistidos com a transação do caso de uso; as filas FIFO usam DLQ correspondente e `maxReceiveCount = 5`.
- No Kind, LocalStack atende em `http://localstack.infra.svc.cluster.local:4566` com serviços SQS e DynamoDB e credenciais fictícias `test/test`; não há conta nem credenciais reais de AWS.
- REST síncrono: cada serviço expõe sua própria API (`/billing`, `/execucao` e API do OS). Acesso local via `kubectl port-forward`; JWT e permissões são validados nos próprios serviços, sem API Gateway ou ALB. Swagger/OpenAPI e a coleção Postman em `docs/Oficina-Mecanica.postman_collection.json` **[PENDENTES]** de verificação/atualização para a Fase 4 (UNR-605).

3. Saga orquestrada

3.1 Escolha e justificativa

Estratégia: orquestração, com o orquestrador no OS Service (`OsSaga`, tabela `os_saga`). Uma saga por OS começa na mesma transação que abre a OS. Decisão registrada na ADR 006 (`docs/adrs/006-orchestrated-saga.md`).

- Um único lugar guarda estado, etapa atual, prazos e o log de compensação. A falha fica legível, testável e demonstrável (`GET /api/ordens-servico/{id}/saga`).
- O ciclo de vida da OS já vivia no OS Service; o orquestrador fica ao lado do agregado que ele conduz.
- Reaproveita o outbox e o SQS FIFO da Fase 3, sem broker novo.
- Coreografia foi rejeitada: a lógica de compensação se espalharia por três repositórios e ciclos de eventos são fáceis de introduzir. RabbitMQ/Amazon MQ foi rejeitado: infraestrutura nova, sem reaproveitar o outbox, custo extra.
- Custo aceito: o OS Service conhece a ordem das etapas (acoplamento do orquestrador).

3.2 Fluxo feliz

Estados do ciclo da OS definidos no plano: `RECEBIDA → EM_DIAGNOSTICO → AGUARDANDO_APROVACAO → AGUARDANDO_PAGAMENTO → EM_EXECUCAO → FINALIZADA → ENTREGUE`; rejeição do orçamento pode retornar a `EM_DIAGNOSTICO` com nova versão do orçamento. `CANCELADA` é terminal para os caminhos de falha. O estado e a etapa da saga são persistidos separadamente da enumeração da OS.

#	Etapa	Comando → destino	Resposta / avanço
1	Diagnóstico	`IniciarDiagnostico` → Execution	`DiagnosticoConcluido`; o mecânico conclui pela API do serviço
2	Orçamento	`GerarOrcamento` → Billing	`OrcamentoGerado`
3	Decisão do cliente	endpoint existente do OS envia `RegistrarDecisao` → Billing	`OrcamentoAprovado` com `checkoutUrl` ou `OrcamentoRecusado`
4	Pagamento	Mercado Pago ↔ Billing	Billing reconsulta o pagamento verificado e responde `PagamentoConfirmado`
5	Reserva de peças	ação local no OS Service	baixa de estoque com controle otimista; falha por estoque insuficiente inicia compensação

#	Etapa	Comando → destino	Resposta / avanço
6	Início do reparo	`IniciarReparo` → Execution	`ReparoIniciado` imediato; `ReparoRecusado` se não houver capacidade/filial válida. Este é o pivô da saga
—	Conclusão	Execution conclui pela API do serviço	`ExecucaoFinalizada` encerra a saga e marca a OS `FINALIZADA`

3.3 Compensações

A compensação roda em ordem inversa, a partir de uma pilha persistida. Se o início do reparo for recusado, as ações anteriores podem ser compensadas. `ReparoIniciado` aceito é o pivô; depois dele, a conclusão do reparo não desfaz etapas anteriores.

Falha	Motivo registrado	O que é compensado
Diagnóstico recusado	`DIAGNOSTICO_RECUSADO`	cancelar execução (no-op se nada foi criado) e cancelar a OS
Orçamento falhou ou não foi gerado no prazo	`ORCAMENTO_FALHOU` / `ORCAMENTO_SEM_RESPOSTA`	cancelar orçamento; se ainda não existir, a resposta pode trazer `orcamentoId` nulo; cancelar a OS
Cliente recusou o orçamento	decisão `RECUSADO`	cancelar orçamento; retornar a OS ao diagnóstico e repetir com a próxima versão do orçamento
Decisão expirou	`DECISAO_EXPIRADA` / `DECISAO_SEM_RESPOSTA`	cancelar orçamento e a OS
Pagamento recusado ou expirado	`PAGAMENTO_RECUSADO` / `PAGAMENTO_EXPIRADO`	cancelar orçamento e a OS
Estoque insuficiente na reserva	`ESTOQUE_INSUFICIENTE`	estornar pagamento, cancelar orçamento e a OS
Execution recusou o reparo	`REPARO_RECUSADO` / `REPARO_SEM_RESPOSTA`	repor estoque, estornar pagamento, cancelar orçamento e a OS

Prazos padrão definidos no plano: diagnóstico 3 dias, resposta automática 5 minutos, decisão 7 dias, pagamento 30 minutos e resposta de compensação 5 minutos. A compensação permite até 5 envios; depois disso, `FALHA_COMPENSACAO` exige tratamento pelo runbook. Os fluxos e motivos ainda precisam de evidência de teste antes da submissão.

3.4 Garantias de consistência

- Efeitos (comandos e ações locais), linha da saga e log de passos são gravados na **mesma transação**; o outbox publica depois do commit.
- Respostas são processadas uma vez (`saga_inbox`) e só aceitas quando `causationId` é igual ao comando aguardado; resposta atrasada ou duplicada não tem efeito.
- Toda espera tem prazo; um poller transforma espera vencida em falha e reenvia compensações não confirmadas, até 5 envios no total, antes de `FALHA_COMPENSACAO` (alerta, resolvida por runbook).
- Um pagamento confirmado **depois** de o orçamento ser cancelado é estornado automaticamente pelo Billing, sem resposta tardia para uma saga encerrada.
- `SAGA_ENABLED` e `SAGA_MENSAGERIA_ENABLED` ficam `false` por padrão. Só o perfil BDD/Kind os ativa depois que bancos e filas estiverem saudáveis; com a saga desativada, permanece o fluxo pass-through da Fase 3 e `GET /api/ordens-servico/{id}/saga` responde 404 antes de acessar o repositório.
- O acesso direto no Kind depende da autenticação JWT e das permissões validadas por cada serviço; não há rota de Lambda/API Gateway no alvo vigente.
- OS criadas antes da saga (sem linha em `os_saga`) seguem o fluxo manual da Fase 3.

3.5 Evidência da saga

Evidência	Situação
Fluxo completo e compensações testados com BDD	**[PENDENTE]** anexar execução, SHA e link verificável
Migração de banco	**[PENDENTE]** evidenciar instalação limpa até V10 e fixture V8 atualizada até V10
Smoke e deploy local no Kind	**[PENDENTE]** anexar job de CI, SHA da imagem, prontidão e health checks
Aceite do Kind como evidência de Kubernetes pelo avaliador	**[CONFIRMAR]** risco externo ainda aberto; não declarar aceite sem confirmação

4. Divisão de responsabilidades e tecnologias

4.1 Por que esta divisão

Serviço	Responsabilidade	Por que separado
OS Service	Abertura, status, histórico e consulta da OS; orquestração da saga	Dono do agregado central e do estado da saga
Billing Service	Orçamento (geração, envio, decisão); pagamento e estorno no Mercado Pago	Integração externa sensível, com segredos e webhook próprios; ritmo de mudança diferente

Serviço	Responsabilidade	Por que separado
Execution Service	Fila de execução, diagnóstico e reparo; avisa a conclusão ao OS Service	Carga orientada a fila e acesso por chave, que casa com NoSQL

A divisão segue os contextos delimitados (atendimento, cobrança, execução) citados no enunciado.

4.2 Tecnologias

Item	OS Service	Billing Service	Execution Service
Linguagem e framework	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.2.5
Banco no alvo local	PostgreSQL 16	PostgreSQL 16 próprio	DynamoDB Local via LocalStack
Mensageria no alvo local	SQS FIFO via LocalStack	SQS FIFO via LocalStack	SQS FIFO via LocalStack
Contexto HTTP	API do OS	`/billing`	`/execucao`
Saúde	Actuator	`/billing/actuator/health`	`/execucao/actuator/health`
Imagem e manifestos	Docker, Kubernetes e Kind efêmero	Docker, Kubernetes e Kind efêmero	Docker, Kubernetes e Kind efêmero

[CONFIRMAR] versões exatas contra os `pom.xml` dos SHAs efetivamente incluídos na entrega.

5. CI/CD, deploy local e evidências de Kind

5.1 Pipelines

O plano prevê pipeline independente por repositório para build, testes, cobertura, qualidade e deploy de validação em um cluster Kind descartável. O fluxo de branches é `feature` → PR para `develop` → integração → `main` como código entregue. O estado real de cada pipeline e proteção de branch só será afirmado com link e SHA verificáveis.

Item	Situação
Pipeline independente de build e testes nos três repositórios	**[PENDENTE]** anexar link e SHA de execução verde por serviço
Proteção de `main` e checagens obrigatórias	**[CONFIRMAR]** ler configuração real nos três repositórios
Cobertura e qualidade no CI	**[PENDENTE]** ver seção 7; não há resultado declarado aqui
Deploy automatizado de validação	Kind efêmero por serviço, **[PENDENTE]** anexar saída de readiness/health e teardown

5.2 Alvo local de integração e demonstração

O alvo efetivo é o cluster Kind `oficina-phase4`, efêmero no CI e descartável no desenvolvimento. A topologia usa namespaces `infra`, `data`, `oficina`, `oficina-billing` e `oficina-execution`; PostgreSQL 16 em serviços separados para OS e Billing; LocalStack para SQS FIFO, DLQs e DynamoDB Local; e imagens carregadas localmente no cluster.

- Nomes de serviço: `oficina-app:8080`, `oficina-billing:8081` e `oficina-execution:8082`.
- Dependências: `postgres-os.data.svc.cluster.local:5432` (`oficina`), `postgres-billing.data.svc.cluster.local:5432` (`billing`) e `localstack.infra.svc.cluster.local:4566` (`sqs,dynamodb`).
- O gate de banco usa uma credencial de migração separada da credencial de runtime: migração limpa até V10 e fixture V8 atualizada até V10. A role de runtime recebe apenas o DML necessário (inicialmente `SELECT/INSERT/UPDATE`); `DELETE` só será concedido se o código provar que é necessário.
- O CI deverá criar o cluster descartável, carregar a imagem, aplicar manifestos, validar prontidão/saúde, guardar a evidência e destruir o cluster.
- Acesso local usa `kubectl port-forward` e validação JWT/roles nos próprios serviços. Não há API Gateway, ALB ou serviço externo de entrada no alvo definido.
- LocalStack usa somente credenciais fictícias em ambiente local. **Não há provisionamento de recursos AWS, deploy em nuvem, Terraform cloud apply nem credenciais AWS nesta estratégia.**

[PENDENTE] anexar o resultado real da integração dos três serviços, o SHA de cada imagem, os health checks e a evidência de teardown. A seleção de Kind não comprova, por si só, o aceite do requisito de deploy Kubernetes pelo avaliador; esse ponto permanece risco de submissão.

6. Observabilidade (New Relic)

[PENDENTE] Esta seção não declara telemetria ativa nem resultados medidos. O plano prevê propagar `traceparent` e `correlationId` e estender a integração New Relic da Fase 3 quando houver chave fornecida fora de banda. Se essa integração não estiver disponível, os testes devem verificar a propagação sem alegar dashboard ou monitoramento ao vivo.

Item exigido no vídeo	Situação
Sinais `saga_prazo_falhou`, outbox `BLOCKED`, idade do item mais antigo, profundidade de DLQ e falha de compensação	**[PENDENTE]** confirmar instrumentação e anexar execução
Rastreamento distribuído com `traceparent` e `correlationId`	**[PENDENTE]** provar propagação entre os três serviços; não declarar New Relic ativo sem evidência
M4: 20 ou mais sagas expiradas distintas sem processar em varreduras consecutivas	**[PENDENTE]** provar limiar e notificação
M7: alerta antes/no limite de cinco envios de compensação ou quando envio bloqueado atingir 20 minutos	**[PENDENTE]** provar limiar e notificação; runbook cobre redrive/reconciliação seguros
Dashboard de saga (volume, duração e falhas)	**[PENDENTE]** anexar dashboard e período observado, se implantado

7. Qualidade e testes (SonarCloud)

[PENDENTE] Este documento não contém percentual de cobertura nem resultado de Quality Gate. A aceitação exige evidência real por serviço; não preencher números por estimativa.

Requisito	Situação
Build e testes independentes	**[PENDENTE]** anexar comando, SHA e resultado de cada repositório
Fluxo completo e compensações com BDD	**[PENDENTE]** anexar execução em Kind; rodar duas vezes para detectar estado/capacidade vazados
Cobertura mínima de 80% por serviço	**[PENDENTE]** anexar resultado medido de JaCoCo/SonarCloud por serviço
Quality Gate no CI	**[PENDENTE]** anexar link, SHA e resultado de SonarCloud

8. Pagamento com Mercado Pago (sandbox)

O Billing Service integra o Mercado Pago por uma camada anticorrupção (`GatewayPagamento`): preferência de checkout, consulta de pagamento, busca por `external_reference` e estorno. A notificação do Mercado Pago nunca é confiada: o pagamento é relido no provedor e `external_reference`, valor e moeda (BRL) são conferidos antes de confirmar; divergência não confirma e estorna.

- Webhook `POST /billing/webhooks/mercadopago` com assinatura HMAC, janela de tempo, deduplicação e limites de corpo e taxa.
- Sem túnel HTTPS aprovado, o webhook real não alcança o Kind; a alternativa suportada é a reconsulta (`POST /billing/pagamentos/{id}/reconsulta`), que usa o mesmo caminho de verificação.
- O gateway fake existe só no perfil de testes e **não** é evidência de pagamento real.
- **[PENDENTE]** Prova no sandbox pelo Rafael via Postman após validar a coleção (UNR-605). Os segredos `MP_ACCESS_TOKEN` e `MP_WEBHOOK_SECRET` são injetados fora de banda e nunca versionados ou registrados em logs.

9. Matriz de rastreabilidade do enunciado

Requisito	Evidência	Situação
Mínimo de 3 microsserviços, repositórios e bancos separados	Seções 2 e 4	Arquitetura especificada; **[PENDENTE]** evidência por SHA
SQL e NoSQL	PostgreSQL 16 e DynamoDB Local, seção 2.2	Alvo especificado; **[PENDENTE]** provar migrações e isolamento
Nenhum serviço acessa o banco de outro	Seção 2.2	**[PENDENTE]** teste de integração/grants
Comunicação REST e mensageria	Seção 2.3	Contratos especificados; **[PENDENTE]** smoke verificável
Saga com compensações e justificativa	Seção 3, ADR 006	Desenho especificado; **[PENDENTE]** execução BDD e falhas
Integração Mercado Pago	Seção 8	**[PENDENTE]** prova sandbox; gateway fake não é prova real
Testes unitários e BDD	Seção 7	**[PENDENTE]** resultado por serviço e fluxo completo
Cobertura mínima de 80% por serviço	Seção 7	**[PENDENTE]** resultado medido por serviço
Qualidade no CI (SonarCloud)	Seção 7	**[PENDENTE]** link e SHA do Quality Gate
Pipelines independentes e `main` protegida	Seção 5.1	**[PENDENTE]/[CONFIRMAR]** configurações e execuções reais
Deploy automatizado em Kubernetes	Seção 5	Kind efêmero **[PENDENTE]** evidência; aceite do avaliador é risco aberto
Observabilidade	Seção 6	**[PENDENTE]** propagação, sinais e alertas verificáveis

Requisito	Evidência	Situação
Swagger ou coleção Postman atualizada	Seção 2.3	**[PENDENTE]** revisão e smoke
Vídeo de até 15 min	Capa	**[PENDENTE]** gravação no alvo local; aceite do Kind continua aberto
PDF com participantes, links, diagrama, saga e justificativas	Este documento	Rascunho

10. Limitações e riscos conhecidos

- **Prazo:** 17/11/2026. BDD, deploy/health no Kind, telemetria, SonarCloud e prova de pagamento ainda precisam de evidências verificáveis.
- **Aceite do Kind:** a seleção do cluster local/efêmero resolve o alvo de integração, mas a aceitação do avaliador como evidência de deploy Kubernetes não está confirmada.
- **Alvo sem nuvem:** não fazem parte do plano vigente deploy em AWS, credenciais de nuvem, cluster gerenciado ou Terraform aplicado em nuvem.
- **Webhook Mercado Pago:** sem túnel HTTPS, a confirmação usa reconsulta, não a notificação real.
- **Compensação:** `FALHA_COMPENSACAO` após cinco envios exige tratamento pelo runbook; não é uma garantia de recuperação automática.
- Este documento não declara cobertura, Quality Gate, deploy concluído, latência, disponibilidade, dashboard ativo ou aceite do avaliador; qualquer resultado deve vir de evidência registrada para um SHA específico.